

732A94 Advanced R programming

Lecture 6, part 2: Parallel programming

Johan Alenlöv (Slides based on Leif Jonssons, Måns Magnusson and Krzysztof Bartoszek)

Linköpings Universitet

- Parallelism
 - What and why
 - Types of parallelism
- Theoretical limits
 - Amdahl's law
 - Gustafson's law
- Parallelism in R
- Load balance and subsetting

Questions?

Any questions from part 1?

Classical introduction: the dining philosophers

Five philosophers, five forks, one bowl each:

```
wait until left fork available
wait until right fork available
eat
release left fork
release right fork
```

Deadlock – everyone picks up their left fork and waits forever.

Solutions:

- synchronize: pick the lower number (Dijkstra)
- wait for permission (mutex)
- release and wait a random time

Mostly we will (a.s.) not be concerned with such problems.

Naive distributed leader election

1. Each processor has a unique id, the processors form a ring, ids can be ordered and are all comparable
2. Each processor sends its id to its left neighbour
3. If the received id is greater than one's own, pass it on to the left neighbour. Otherwise the receiving processor becomes silent
4. If a processor receives its own id back, it declares itself the leader

Complexity is $O(n^2)$ messages passed.

The same idea gives a distributed minimum/maximum algorithm.

What is parallelism?

- Multiple cores
- Each core works with its own part
- Cores can exchange information

```
parallel::detectCores()
```

```
## [1] 8
```

Why parallelism?

- Single-core limits (clock speed stopped growing around 2005)
- Handling larger data
- Solving problems faster
- More robust – if one processor fails, the others can still work
- More and more important

Is there any but...?

- Multicore systems (one machine, shared memory)
- Distributed systems (many machines, message passing)
- Graphical processing units (GPU)

Which one you have decides what is cheap: sharing a 2 GB matrix between threads is free, sending it over a network is not.

Strong scaling: Amdahl's law

- Deals with a fixed problem size and increasing resources

Weak scaling: Gustafson's law

- Deals with an increasing problem size along with increasing resources

Speedup:

$$S_P = \frac{\text{execution time on 1 processor}}{\text{execution time on } P \text{ processors}} \quad S_P \leq \frac{1}{f_s + \frac{f_p}{P}}$$

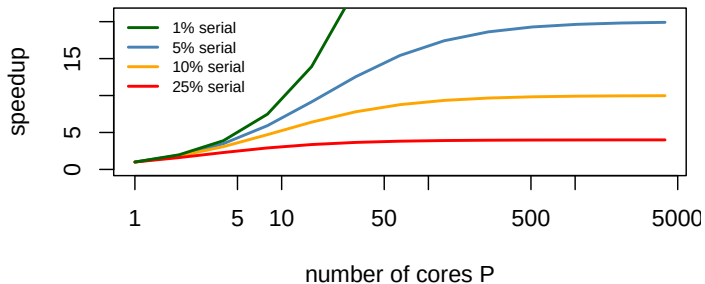
f_s = serial fraction of the computations, f_p = parallel fraction, P = number of cores.

For a fixed size problem the single-core computation time is fixed = 1.

Efficiency:

$$S_P P^{-1} = \frac{\text{execution time on 1 processor}}{P \cdot (\text{execution time on } P \text{ processors})}$$

Amdahl's law



5% serial code caps you at a speedup of 20, no matter how many cores.

Speedup:

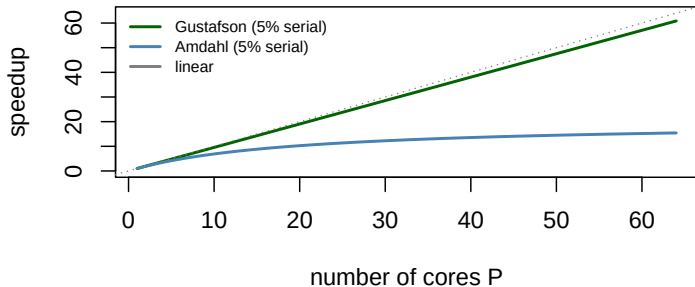
$$S_P \leq \frac{\alpha + (1 - \alpha)P}{\alpha + (1 - \alpha)} = P - \alpha(P - 1)$$

where α = fraction of time dedicated to serial computations and P = number of cores.

Here the problem size scales with P , and the *parallel* execution time is fixed = 1. If we only had one core, the P parallel computations would have to be done on that core, taking time $(1 - \alpha)P$.

Amdahl asks “how much faster?”; Gustafson asks “how much more can I do?”.

Amdahl vs. Gustafson



Costs of parallelism:

- communication
- load balancing (NP-hard)
- scheduling (NP-hard, but there is a $4/3$ -approximate algorithm)

Symmetry: if every processor sees the same thing, no one can break the tie.

Byzantine generals problem: the network must reach a decision, but some nodes can be corrupt.

Fine-grained vs. **embarrassingly parallel** (EP).

Parallelism in R (embarrassingly parallel)

Based on `lapply()`: the iterations inside the loop are independent of each other (H. Wickham, *Advanced R*, p. 201).

What about

```
fctl <- 1  
for (i in 1:n) fctl <- i * fctl
```

Not independent – iteration i needs the result of iteration $i - 1$. This is a *reduction*, not a map. (It can still be parallelised, but not by splitting the loop.)

Two approaches:

1. `mclapply()`
2. `parLapply()`

Plus the **par*** family: `parSapply()`, `parApply()`, `parRapply()` (rows), `parCapply()` (columns), and the load-balancing variants `parLapplyLB()`, `parSapplyLB()`.

Simple to use

Low overhead (startup)

Does not work on Windows

Only multicore (one machine)

```
library(parallel)
res <- mclapply(1:1000, f, mc.cores = detectCores())
```

```
parLapply(type = "PSOCK")
```

Works everywhere

Good for testing and development

Slow across multiple nodes

```
cl <- makeCluster(detectCores(), type = "PSOCK")  
res <- parLapply(cl, 1:1000, f)  
stopCluster(cl)
```

Always `stopCluster()` – otherwise the worker R processes stay alive.

```
parLapply(type = "MPI")
```

Good for multiple computers

Good for production

Cannot really be used interactively

Needs the **Rmpi** package

This is what you use on a cluster like NSC.

parSapply: a new R on each core

- Variables need to be copied **explicitly**
- A new random seed on each core
- All code needs to be sourced on each core

```
cl <- makeCluster(2)
clusterExport(cl, c("my_data", "my_helper"))
clusterEvalQ(cl, library(MASS))
parSapply(cl, 1:10, function(i) my_helper(my_data, i))
stopCluster(cl)
```

`clusterExport()` is the number one source of “object not found” in parallel R code.

A worked example

```
library(parallel)
euclidean <- function(a, b) if (b == 0) a else
  euclidean(b, a %% b)
cl <- makeCluster(2)
clusterExport(cl, "euclidean") # <- do not forget!
res <- parLapply(cl, 1:1000, function(i) euclidean(i, 1000))
stopCluster(cl)
head(unlist(res), 12)
```

```
## [1] 1 2 1 4 5 2 1 8 1 10 1 4
```

Without the `clusterExport()` line this fails with “could not find function *euclidean*”.

See `parallel_example.R` and `parallel_scalarproduct.R`

Random numbers in parallel

Each worker starts with its *own* seed – so `set.seed()` in the parent does **not** make a parallel run reproducible.

```
RNGkind("L'Ecuyer-CMRG")  
cl <- makeCluster(2)  
clusterSetRNGStream(cl, iseed = 1)  
unlist(parLapply(cl, 1:4, function(i) round(rnorm(1), 3)))
```

```
## [1]  0.461  1.343 -0.485 -0.626
```

```
stopCluster(cl)
```

For `mclapply()` use `mc.set.seed = TRUE` with the same RNG kind. This matters for every simulation study you will ever run.

`lapply(list, ...)` works in the order of the list, which can leave nodes idle.

- Load balancing is NP-hard
- Often we do not know the running time for `list[[i]]`
- Submit to cores in order? Split into consecutive equal chunks?

Example: all 2^p regression models on p predictors

- generate all combinations: $\emptyset, \{1\}, \dots, \{p\}, \{1, 2\}, \dots$
- the later ones are much more expensive – **randomize the order**

R's `par` family has load balancing: `par[L/S]applyLB`.

Subset methods (non-EP): many observations

`glm()` with a large number of observations – **chunk averaging** (estimation):

- break the data into chunks of rows
- apply `glm()` to each chunk, in parallel
- average the results to obtain a single estimate

Requires the observations to be i.i.d. and the model to be decent; asymptotic equivalence for large samples.

If the chunks are not identically distributed (first all the cases, then all the controls) – **randomly permute** the observations first. That will not harm an already random arrangement.

Subset methods (non-EP): many predictors

`glm()` with a large number of predictors – **subsetting variables** (prediction, decent model):

- create random subsets of predictors
- apply `glm()` to each subset
- predict from each subset
- combine the predictions, e.g. by averaging or a majority rule

Prediction is difficult in high dimensions – the curse of dimensionality.

```
library(future); library(furrr)
plan(multisession, workers = 2)
head(future_map_dbl(1:1000, \(i) euclidean(i, 1000)), 12)
```

```
## [1] 1 2 1 4 5 2 1 8 1 10 1 4
```

```
plan(sequential)
```

No `clusterExport()` – `future` works out what the expression needs and ships it for you. For random numbers pass `.options = furrr_options(seed = TRUE)`.

`future` picks the backend at *run* time:

```
plan(sequential)           # debugging
plan(multisession, workers = 4) # this laptop
plan(cluster, workers = c("n1", "n2")) # other machines
```

The code you wrote does not change – only the `plan()` line does. So the same script runs on your laptop and on NSC.

`furrr` mirrors `purrr`, `doFuture` bridges to `foreach`.

Questions?

See you next time!